

Guide Démo

GitHub Desktop + VS Code | Formation DWWM

ETAPE 1 — Créer le repo sur GitHub

STAGIAIRES

Je suis Alice. Je crée le projet sur GitHub et j'invite mon équipier Bob. On commence toujours par créer le repo en ligne avant de toucher à quoi que ce soit en local.

GITHUB WEB — GITHUB.COM/NEW

1. Repository name : `mon-projet-duo`
2. Visibilité : **Public**
3. Cocher "**Add a README file**" — indispensable !
- 4.. **Gitignore** Template : choisir **Node**
5. Cliquer "**Create repository**"

GITHUB DESKTOP — CLONER LE REPO

1. File > **Clone repository**
2. Onglet **GitHub.com** — trouver `mon-projet-duo`
3. Choisir le dossier local puis **Clone**

Montrer : "Current Branch : main" apparaît en haut de GitHub Desktop après le clone.

PIEGE A EXPLIQUER

Sans "Add README" coché, le repo est vide — pas de commit initial, pas de branche `main`.
GitHub Desktop affiche une erreur au clone. **Toujours cocher cette case.**

ETAPE 2 — Inviter Bob comme collaborateur

CE QUE TU DIS

Bob peut lire le repo mais pas écrire dedans. Sans invitation, son push sera refusé avec une erreur 403 Forbidden. Il faut l'ajouter comme collaborateur.

GITHUB WEB — SETTINGS DU REPO

1. Sur la page du repo — onglet **Settings** dans la barre en haut
2. Menu gauche — **Collaborators**
3. Bouton "**Add people**"
4. Taper le pseudo GitHub de Bob — confirmer l'ajout

Montrer : La page affiche "Invitation sent" avec l'email de Bob en attente.

BOB — ACCEPTER L'INVITATION

1. Bob reçoit un email — cliquer "**View invitation**"
2. Ou aller sur `github.com/notifications`
3. Cliquer "**Accept invitation**"

PIEGE CLASSIQUE

Settings du repo (dans la barre d'onglets du repo) est différent du **Settings du compte** (avatar en haut à droite). Erreur très fréquente chez les débutants.

ETAPE 3 — Bob clone le repo

CE QUE TU DIS

Maintenant je suis Bob. J'ai accepté l'invitation. Je clone le projet sur ma machine. Le clone copie tout l'historique Git en local.

GITHUB DESKTOP — BOB CLONE

1. File > **Clone repository**
2. Onglet GitHub.com — chercher `alice-dev/mon-projet-duo`
3. Choisir un dossier différent du dossier d'Alice
4. Clone — Bob est sur `main`

Montrer : Montrer les deux fenêtres GitHub Desktop cote à cote — même repo, même état initial.

VSCODE — OUVRIR LE PROJET

1. Dans GitHub Desktop — bouton **"Open in Visual Studio Code"**
2. VS Code s'ouvre avec le dossier du projet
3. En bas à gauche : la branche courante `main` est visible

CHECKPOINT

Alice et Bob ont tous les deux le projet en local sur `main`. Étape suivante : chacun crée sa propre branche avant de coder.

ETAPE 4 — Créer les branches

CE QUE TU DIS

Règle absolue : on ne code jamais directement sur main. Chacun crée une branche pour sa fonctionnalité. On peut travailler en parallèle sans se bloquer.

ALICE — GITHUB DESKTOP

1. Cliquer sur **"Current Branch : main"** en haut
2. **"New Branch"**
3. Nom : `feature/navbar`
4. **"Create Branch"**

BOB — GITHUB DESKTOP (DE SON COTE)

1. Même procédure
2. Nom : `feature/login`
3. **"Create Branch"**

VSCODE CONFIRME LE CHANGEMENT

1. En bas à gauche de VS Code : `main` est passé à `feature/navbar`
2. GitHub Desktop et VS Code sont synchronisés en temps réel

Montrer : Moment fort : le badge de branche en bas de VS Code change en même temps que dans GitHub Desktop.

CONVENTION DE NOMMAGE

`feature/` nouvelle fonctionnalité | `fix/` correction | `chore/` config — toujours en minuscules avec tirets, sans espaces.

ETAPE 5 — Coder et commiter dans VS Code

CE QUE TU DIS

Je crée mon fichier Nav bar dans VS Code. Regardez GitHub Desktop en même temps — il détecte le fichier automatiquement dès la sauvegarde. C'est le moment le plus parlant de toute la démo.

VSCODE — CREER LE FICHIER

1. Créer le dossier `src/` dans l'explorateur VS Code
2. Créer le fichier `src/Navbar.jsx`
3. Taper ce code minimal :

```
export default function Navbar () {  
  return <nav>Ma Navbar</nav>  
}
```

4. **Cmd+S** (Mac) ou **Ctrl+S** (Windows) pour sauvegarder

Montrer : Passer IMMEDIATEMENT dans GitHub Desktop après la sauvegarde.

GITHUB DESKTOP — DETECTION AUTOMATIQUE

1. Panneau gauche : `src/Navbar.jsx` apparaît avec un + vert
2. Panneau droit : le diff montre les lignes ajoutées en vert
3. Champ **Summary** en bas : taper `feat : ajout composant Navbar`
4. Bouton bleu **"Commit to feature/navbar"**
5. Bouton en haut **"Publish branch"** = git push

Montrer : Montrer le diff vert — c'est ce que Bob verra dans la PR pour relire le code.

CONVENTION MESSAGES DE COMMIT

`feat` : nouvelle fonctionnalité | `fix` : correction | `style` : mise en forme | `docs` : documentation
| `refac`: refactoring

ETAPE 6 — Ouvrir la Pull Request

CE QUE TU DIS

J'ai pushe ma branche. Je demande à Bob de relire mon code avant de le fusionner dans main. C'est ça une Pull Request — une demande formelle de validation.

GITHUB DESKTOP — CREER LA PR EN 1 CLIC

1. Après le push, une bannière apparaît : **"Create Pull Request"**
2. Cliquer dessus — GitHub s'ouvre sur le formulaire automatiquement

Montrer : La bannière disparaît après quelques secondes — cliquer vite ou Branch > Create Pull Request.

GITHUB WEB — REMPLIR LE FORMULAIRE

1. Titre : `feat: ajout navbar responsive`
2. Description : ce qui a été fait, pourquoi
3. Reviewers — cliquer l'engrenage — ajouter **bob-dev**
4. Bouton vert **"Create pull request"**

Montrer : Montrer : "base : main <- compare : feature/navbar". Expliquer le sens de la flèche.

CHECKPOINT

La PR est ouverte. Bob reçoit une notification. `main` n'a PAS encore change — le code d'Alice est en attente de validation.

ETAPE 7 — Review, Merge et Pull

CE QUE TU DIS

Je suis maintenant Bob. J'ai reçu la notification. Je relis le code d'Alice. On ne merge jamais sans relire — même en formation.

BOB RELIT LA PR — GITHUB WEB

1. Aller sur la PR — onglet **"Files changed"**
2. Lire les lignes vertes (+) ajoutées, rouges (-) supprimées
3. Cliquer sur le + d'une ligne pour laisser un commentaire inline
4. Bouton **"Review changes"** — Approve — **"Submit review"**

Montrer : Faire un vrai commentaire en live — les stagiaires voient l'utilité du code review.

ALICE MERGE — GITHUB WEB

1. Après approbation de Bob — bouton vert **"Merge pull request"**
2. **"Confirm merge"**
3. **"Delete branch"** — garder le repo propre

LES DEUX — RECUPERER MAIN A JOUR (GITHUB DESKTOP)

1. Switcher sur la branche `main` dans GitHub Desktop
2. Bouton **"Fetch origin"** puis **"Pull origin"**
3. VS Code se met à jour — `Navbar.jsx` apparaît dans l'arborescence

Montrer : Moment final : `Navbar.jsx` est dans VS Code sur main. Il est arrivé via le merge.

WORKFLOW COMPLET !

Le cycle complet : **branch > commit > push > PR > review > merge > pull** — C'est ce que les stagiaires répèteront tous les jours en entreprise.